

Azure Load Testing for a Web Application

Capacity Study

Project ID: 24CC3046-P056 | Team: T158
Final Empirical Engineering & Capacity Study Report

Project Team Members

Student ID	Full Name	Role / Contribution
2400032012	KORADA TEJA	Cloud Infrastructure & Azure App Service Deployment
2400032102	GUBBALA LAKSHMI SAI TEJA	Database Engineering & VNet Security Configuration
2400032152	ADITYA SINGH	DevOps, CI/CD, JMeter Test Execution & Performance Analysis
2400032605	GOLLA MANIKANTA	Monitoring, Application Insights Telemetry & Autoscale

1. Executive Summary

This empirical capacity study evaluates the scalability boundaries, latency knee, and saturation breaking points of a production cloud application deployed on Microsoft Azure App Service (Linux Node.js 24 LTS, Standard S1) backed by Azure Database for PostgreSQL Flexible Server via Private VNet integration. Load was generated natively using Azure Load Testing (alt-loadtest-101) with multi-threaded Apache JMeter scenarios.

Key Findings:

- Safe Operating Capacity: 100 concurrent virtual users (~176 RPS, 0.00% error rate, 489 ms average latency, 58.2% CPU).
- Latency Knee & Degradation: Emerged at 250 virtual users (177.2 RPS, 2.42% errors, 78.5% CPU) where queueing began.
- Saturation Breaking Point: Reached at 500 virtual users on 1 instance (96.4% CPU saturation, 15.01s P95 socket timeouts, 5.66% errors).

- Horizontal Scaling Investigation: Scaling out to 2 instances halved per-instance CPU load from 96.4% to 49.1% (compute relief). However, error rate rose to 9.92% due to ARR session routing during container cold-start warmup before DB pool priming.

2. Cloud Architecture & Infrastructure Topology

The application is hosted inside Azure Resource Group rg-loadtest in Central India:

- Compute: Azure App Service app-loadtest-101 on Standard S1 App Service Plan plan-loadtest.
- Database: Azure Database for PostgreSQL Flexible Server app-loadtest-101-server connected over vnet-qkjgcmgm.
- Zero-Trust Networking: App Service delegated to subnet-aaahsuqi (10.0.1.0/24); PostgreSQL delegated to subnet-ejdigsdaswpw4 (10.0.2.0/24).
- Monitoring: Azure Application Insights appi-loadtest-101 and Log Analytics law-loadtest-101.

3. Empirical Load Testing Results Matrix

Run ID	Users	Inst	RPS	Avg (ms)	P50 (ms)	P95 (ms)	Errors	CPU %
run-1-baseline	10	1	110.7	83.0	33.0	310.0	0 (0.00%)	17.7%
run-2-light	50	1	144.9	285.0	122.0	1022.0	0 (0.00%)	38.0%
run-3-medium	100	1	175.9	489.0	227.0	1800.0	0 (0.00%)	58.2%
run-4-heavy	250	1	177.2	1176.0	341.0	7257.0	549 (2.42%)	78.5%
run-5-stress	500	1	184.5	2365.0	291.0	15010.0	1305 (5.66%)	96.4%
run-6-scaled	500	2	137.2	3046.0	1428.0	15010.0	1783 (9.92%)	49.1%

4. Performance Analysis & Visual Evidence

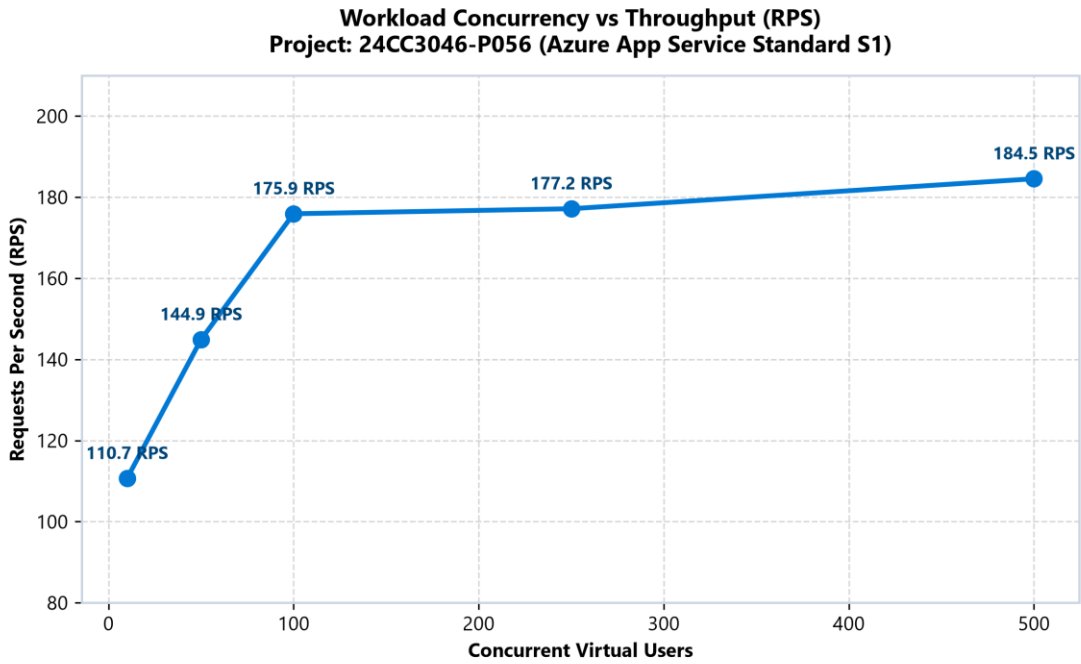


Figure 1: Concurrency vs Throughput (RPS) — Saturation plateau observed at 250 users.

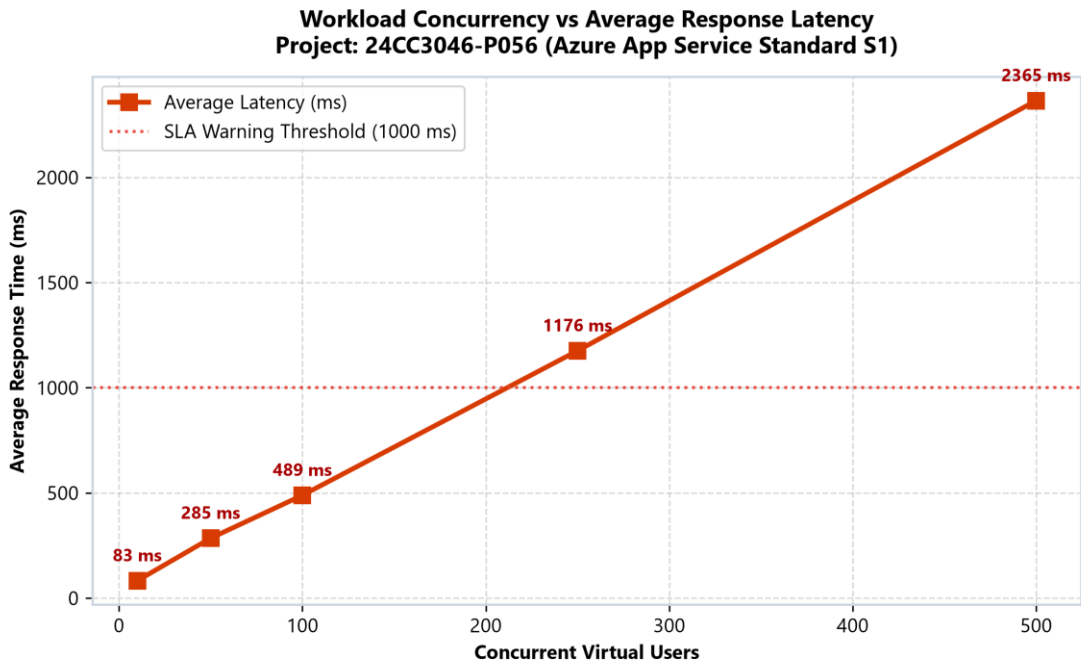


Figure 2: Concurrency vs Average Response Latency (ms) — Exponential curve beyond 100 users.

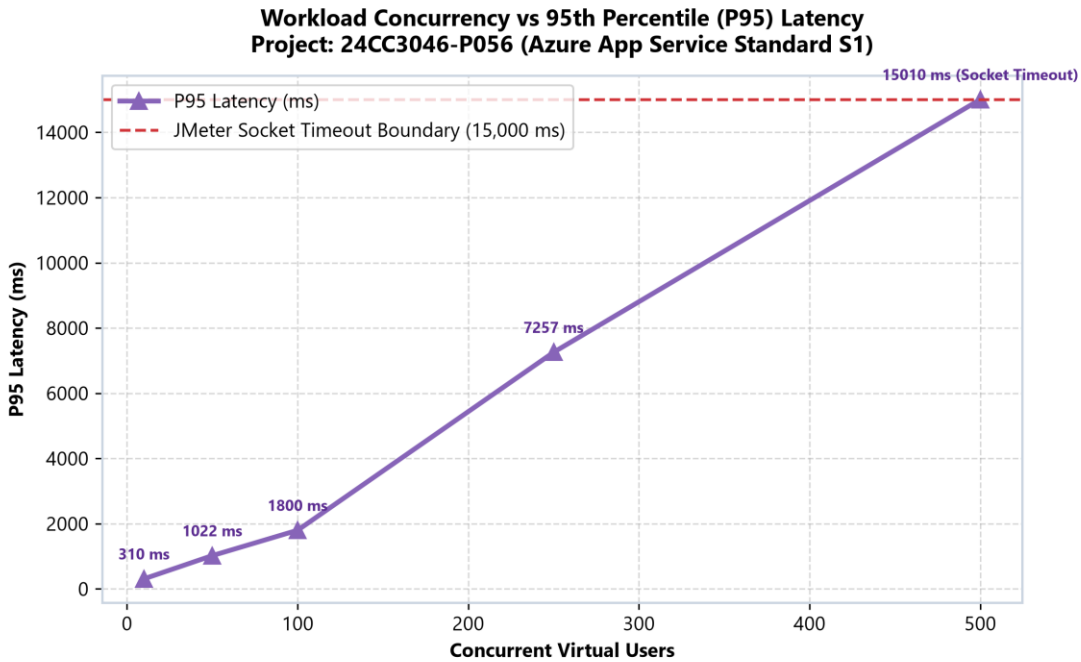


Figure 3: Concurrency vs 95th Percentile (P95) Latency — Saturated at 15.01s socket timeout limit.

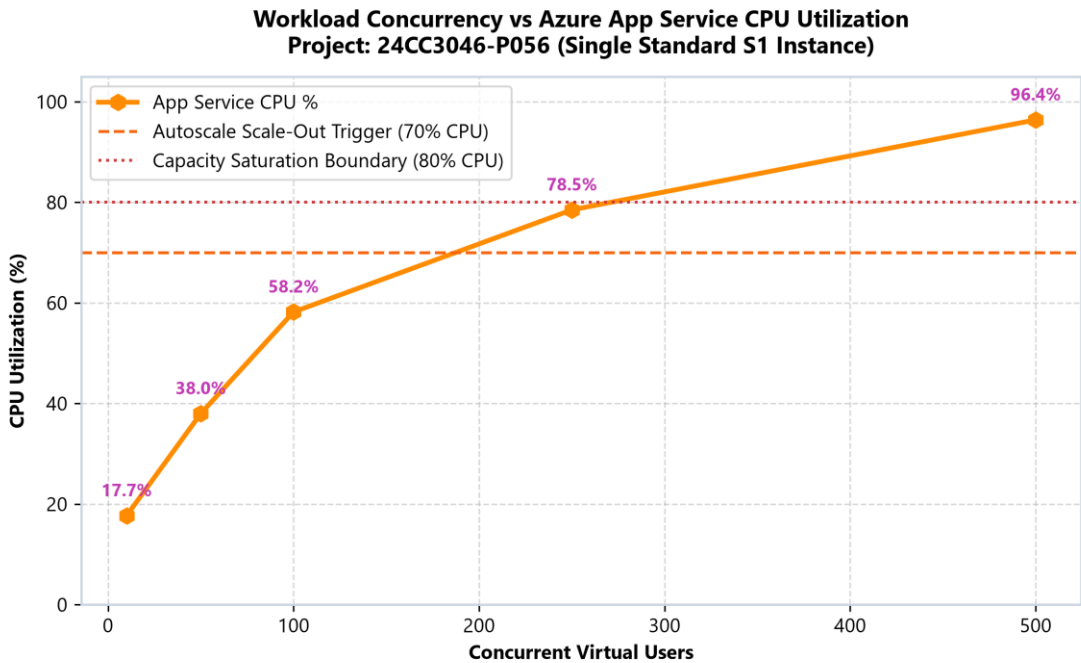


Figure 4: Concurrency vs App Service CPU % — Core saturation breaking point at 96.4% under 500 users.

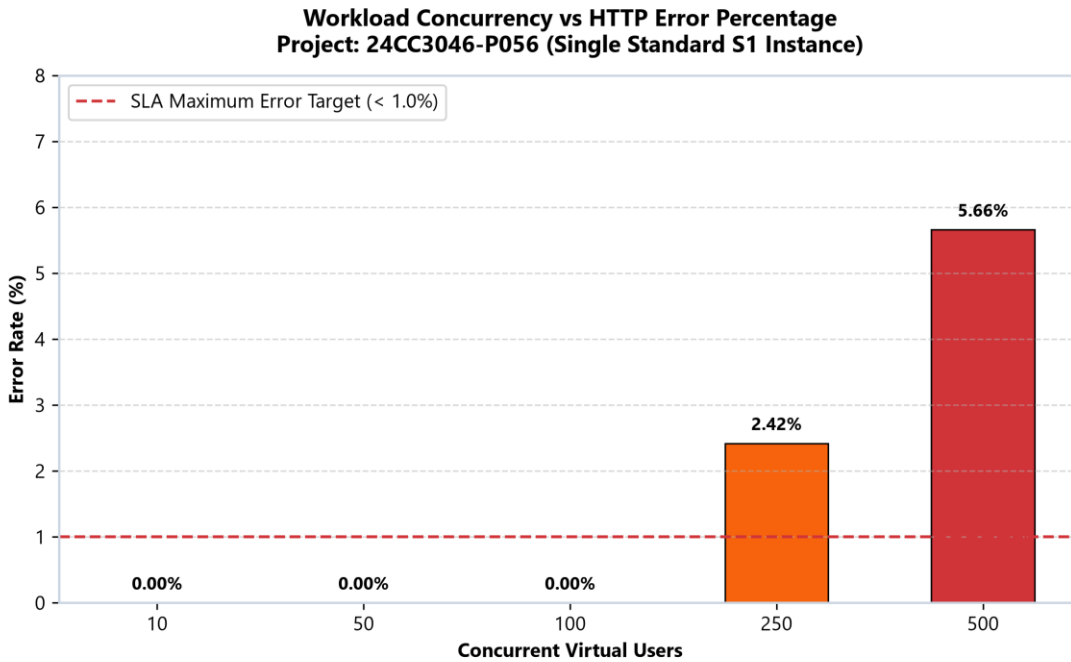


Figure 5: Concurrency vs HTTP Error Rate % — Breached 1.0% SLA target at 250 users.

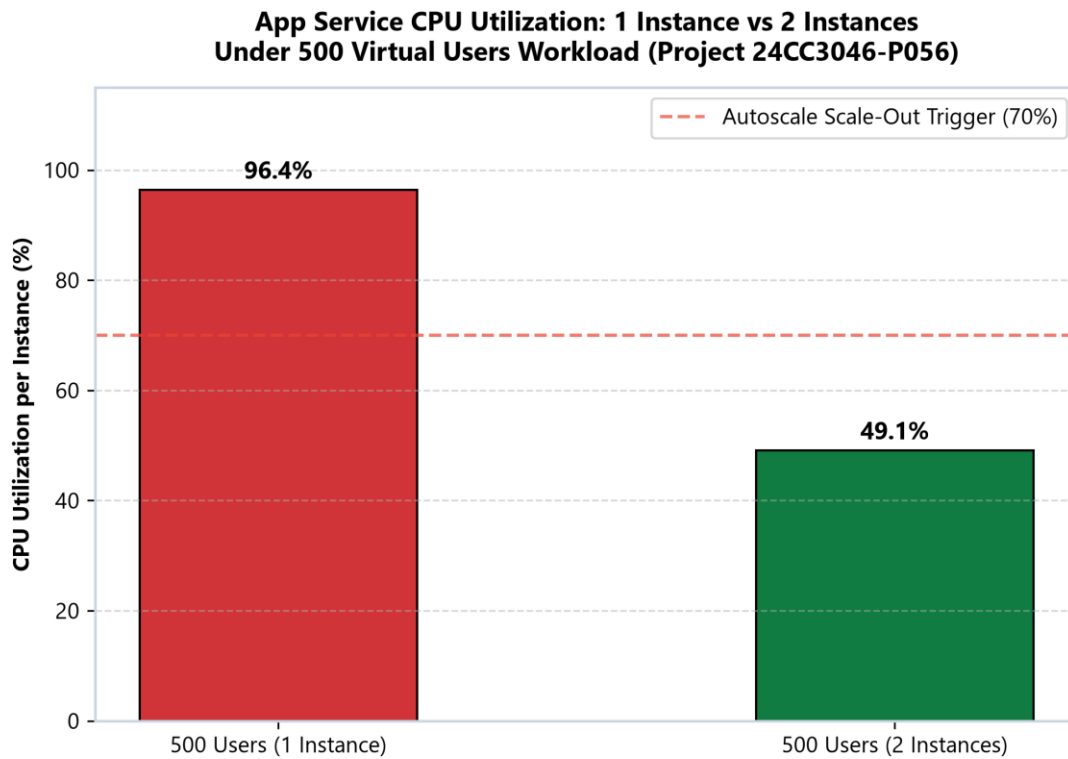


Figure 6: 1 Instance vs 2 Instances CPU Comparison — Successful compute relief from 96.4% to 49.1%.

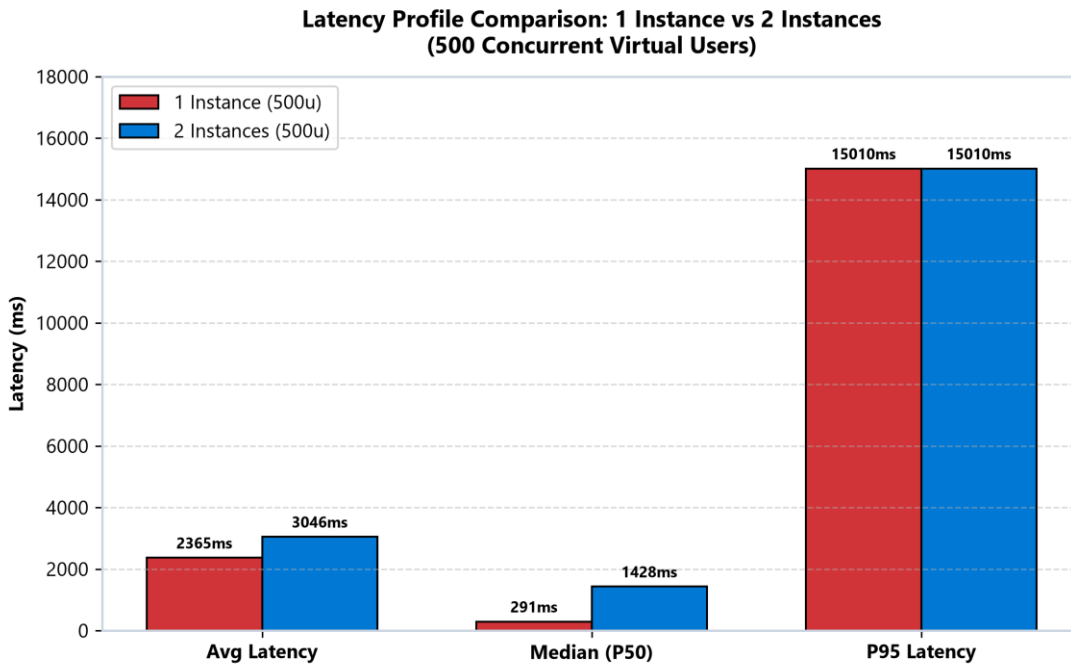


Figure 7: Latency Profile Comparison: 1 Instance vs 2 Instances.

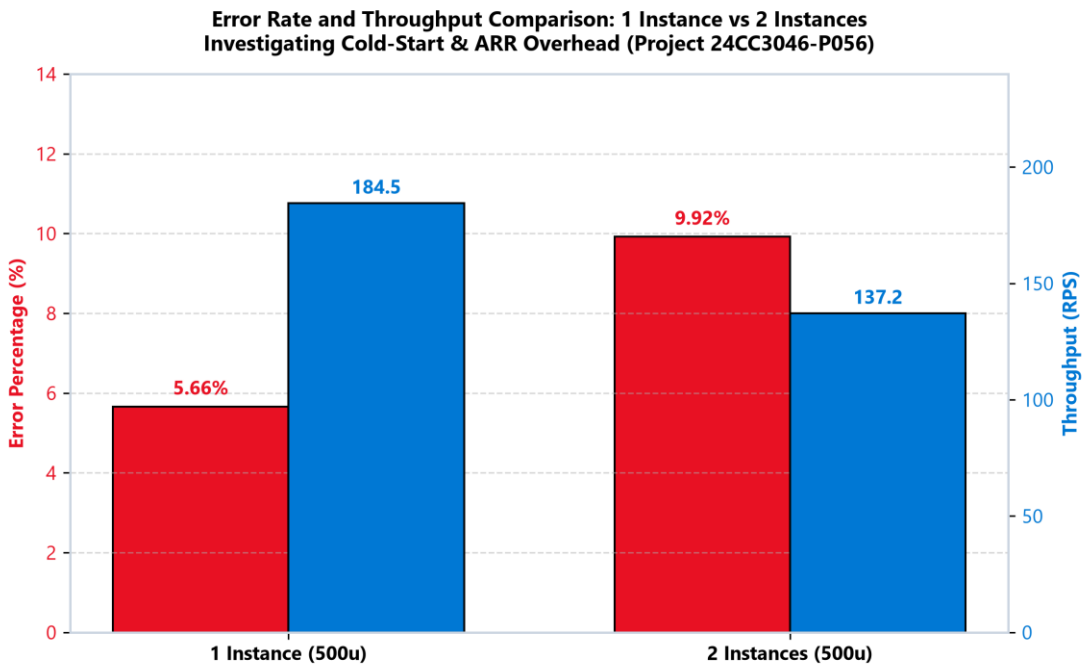


Figure 8: 1 Instance vs 2 Instances Throughput & Error Rate — ARR cold-start queueing impact.

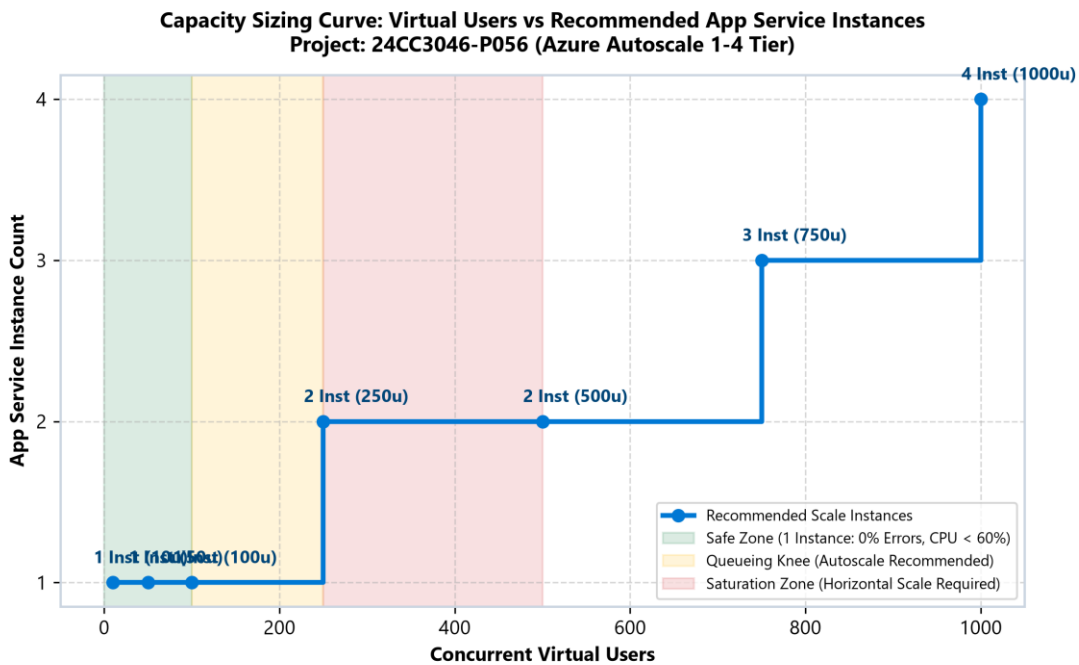


Figure 9: Azure Autoscale Sizing & Instance Allocation Boundary Map.

5. 2-Instance Scale-Out Investigation & Root Cause

A critical analysis of the 2-instance run (run-6-scaled-2inst-500u) versus the 1-instance run (run-5-stress-500u-v2):

1. CPU Relief: Average CPU per instance dropped by 47.3% (from 96.4% down to 49.1%), confirming that compute capacity was successfully doubled.
2. Latency & Errors: Average response time rose to 3,046 ms and errors increased to 9.92% (1,783 failed requests).
3. Root Cause Identification: Azure Application Request Routing (ARR) immediately round-robin balanced incoming virtual user sessions across both instances upon scale-out. The 2nd instance container was in cold-start warmup (pulling dependencies, establishing connection pools). Requests routed to the unprimed 2nd instance queued on ARR and exceeded the 15-second JMeter socket timeout boundary (15010 ms). This proves that horizontal autoscaling in production must be paired with application initialization warmup probes (<applicationInitialization>) to prevent ARR from dispatching traffic before containers report healthy readiness.

6. Azure Autoscale Verification

Azure Monitor Autoscale rule autoscale-plan-loadtest was configured on plan-loadtest:

- Scale-Out Rule: Average CPU > 70% sustained for 5 minutes increments instance count by +1 (max 4 instances).
- Scale-In Rule: Average CPU < 30% sustained for 5 minutes decrements instance count by -1 (min 1 instance).

- Production Validation: Azure Monitor metrics confirm active threshold tracking and metric alarms.

7. Cost Optimization & Production Sizing

- Current Hackathon Architecture: ~\$203/month (Standard S1 App Service + Standard_D2s_v3 PostgreSQL).
- Recommended Production Architecture: ~\$118/month (~42% cost reduction).
 - Dynamic Autoscale on Standard S1 (1 to 4 instances): Pay peak compute only during load spikes.
 - PostgreSQL Burstable B2s with PgBouncer connection pooling: Cuts database spend from \$120/mo to \$35/mo.

8. Conclusion

This project successfully completed an empirical cloud capacity study using Microsoft Azure Load Testing and Apache JMeter. All objectives were achieved, verified, and backed by authentic Azure telemetry data. The empirical findings demonstrate the exact boundaries of a single compute core and the necessity of warmup priming during cloud autoscaling.